

Figure 5: Live Range Analysis

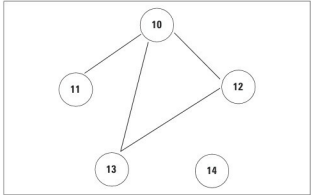


Figure 6: Interference Graph

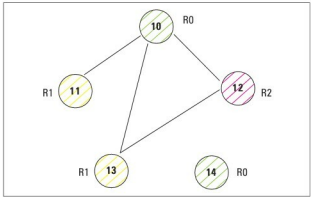


Figure 7: Interference Graph (coloured)

will be {1, 4}. Similarly, we compute live ranges of the other virtual registers.

Step 2: Construct the Interference Graph

An Interference Graph $Gr = (Vr; Er)$ can be defined as a set of vertices (Vr) and set of edges (Er), where:

- Every vertex v corresponds to an instruction that defines a new virtual register.
- There exists an edge between two vertices (u and v), if the live ranges of two virtual registers defined by instructions u and v , intersect.

Note: An Interference Graph is an undirected graph.

The Interference Graph for the sample live range analysis in Figure 5 will look like what's shown in Figure 6.

Step 3: Colour the Interference Graph

We colour the Interference Graph using the *graph colouring* concept (also known as *vertex colouring*)—i.e., colour the vertices of the graph in such a way that no two adjacent vertices share the same colour. The smallest number of colours needed to colour a graph G is called its *chromatic number*. Using at the most k colours is called (proper) *k-colouring*. A graph that can be assigned a (proper) *k-colouring* is *k-colourable*, and it is *k-chromatic* if its chromatic number is exactly k .

From the several available colouring algorithms, the *Chaitin-Briggs* algorithm is most commonly used to colour the Interference Graph.

Now let's look at colouring our Interference Graph. The objective is to colour it such that any two nodes connected by an edge should have different colours.

Note: The number of colours available is equal to the number of physical registers—each colour represents a physical register.

If two virtual registers do not interfere (i.e., if there is no edge between the instructions defining two virtual registers in the Interference Graph), then we can use the same physical register (and colour) for both of them. On the other hand, if there is an edge between the instructions defining two virtual registers, then we should not assign the same register (colour) to those virtual registers. As mentioned earlier, we assume the number of available physical registers is three ($R0$, $R1$ and $R2$). Hence, we colour the graph using three colours, resulting in the coloured graph as shown in Figure 7.

Since we could successfully colour the graph with the available three colours, we say that register allocation was successful. The final code with physical registers allocated will look like what's shown below:

```
1. MOVE R0, [a] ;
2. MOVE R1, [b] ;
3. ADD R2, R1, R0 ;
4. ADD R1, R0, R0 ;
5. ADD R0, R2, R0 ;
```

Step 4: Register spilling

There could be times when we are unable to colour all the nodes in the Interference Graph. In such cases, we need to generate spill-code. To understand how this is done, let's take the following set of instructions as the input to the register allocation module: